

Portable MAW–ECM Blueprint

Algorithmic notes for moving the implementation to another ROM codebase

RVE 2D MAW–ECM implementation notes

June 3, 2026

1 Purpose

This document describes the MAW–ECM construction in a form that is intended to be copied to another ROM workbench, for example a Burgers 2D ROM code. The emphasis is on the algebra, the data contracts, and the algorithmic sequence. The concrete Kratos stage names are mentioned only as implementation references.

The method starts from a standard, fixed ECM rule and transforms it into a manifold-adaptive cubature rule. Instead of one fixed weight vector, the final offline object is

$$q \mapsto w(q),$$

where q is the reduced/manifold coordinate used online. The support remains sparse, but the weights vary over the sampled manifold.

2 Notation

Symbol	Meaning
n_e	number of full mesh elements/cubature entities.
N_s	number of sampled manifold states/nodes.
$q_j \in \mathbb{R}^{n_q}$	reduced/manifold coordinate of sample j .
c	channel label, e.g. residual, strain, stress, output quantity.
$Z_{\text{ini}}^c \subset \{1, \dots, n_e\}$	initial ECM support for channel c .
$n_c = Z_{\text{ini}}^c $	number of initial ECM candidates.
$w_{\text{ini}}^c \in \mathbb{R}^{n_c}$	initial fixed ECM weights on Z_{ini}^c .
$A_j^c \in \mathbb{R}^{m_c \times n_c}$	local constraint matrix at manifold sample j .
$b_j^c \in \mathbb{R}^{m_c}$	local target at manifold sample j .
$W^c \in \mathbb{R}^{n_c \times N_s}$	adaptive training weights after MAW pruning.
Z_{red}^c	final MAW support for channel c .
$K_{\text{graph}} \in \mathbb{R}^{N_s \times N_s}$	graph Laplacian over manifold samples.

Channel. A channel is any quantity for which we want a cubature rule. In the 2D RVE code, we currently use:

$$c \in \{\text{res}, \text{eps}, \text{sig}\}.$$

In a Burgers 2D workbench, typical channels could be:

$$c = \text{residual}, \quad c = \text{lift}, \quad c = \text{drag}, \quad c = \text{energy},$$

or any other reduced quantity assembled as an element sum.

3 Required data contract

For every channel c , MAW-ECM needs:

1. A fixed ECM support Z_{ini}^c and weights w_{ini}^c .
2. For each sampled manifold state q_j , a local matrix

$$A_j^c \in \mathbb{R}^{m_c \times n_c}.$$

Each column corresponds to one element in Z_{ini}^c .

3. A target vector $b_j^c \in \mathbb{R}^{m_c}$.
4. Optionally, a graph over the samples, represented by K_{graph} .

The local MAW exactness condition is:

$$A_j^c w_j^c = b_j^c, \quad j = 1, \dots, N_s.$$

How b_j^c is built in the current implementation. The current Stage8b implementation uses the ECM anchor:

$$b_j^c = A_j^c w_{\text{ini}}^c.$$

This means MAW preserves the fixed ECM rule exactly on the initial ECM support. If a different codebase has a robust full-order target available, it can instead use:

$$b_j^c = \sum_{e=1}^{n_e} a_{j,e}^c,$$

or the equivalent full integration vector. The rest of the MAW pruning algorithm does not change.

4 Classical ECM bootstrap

The first step is always a standard ECM rule. MAW-ECM does not start from the full mesh; it starts from a fixed compressed ECM rule.

For a channel c , let the global snapshot matrix be

$$Q^c = \begin{bmatrix} | & | & \cdots & | \\ q_1^c & q_2^c & \cdots & q_{n_e}^c \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{M_c \times n_e},$$

where each column is the contribution of one element/cubature entity to the quantity being compressed. The full target is

$$b_{\text{full}}^c = Q^c \mathbf{1}.$$

Classical ECM computes a sparse nonnegative approximation

$$Q^c w_{\text{full}}^c \approx b_{\text{full}}^c, \quad w_{\text{full},e}^c = 0 \quad \text{for } e \notin Z_{\text{ini}}^c.$$

In our current code this is handled by:

```
_compute_classic_res_bootstrap,    _compute_classic_hom_bootstrap.
```

The bootstrap uses deterministic SVD by default:

```
--res-bootstrap-rsvd-randomized 0.
```

The flag name is historical; the default path is deterministic. Use randomized SVD only for experiments where exact reproducibility is not important.

4.1 Sum of weights in the bootstrap

For classical ECM we normally enforce:

$$\mathbf{1}^T w = S_c,$$

where S_c is usually the total number of elements or the full integration measure, depending on the channel definition. In the 2D RVE examples:

$$S_c = 990.$$

This is not only a cosmetic condition. It fixes the integral of constant fields. For many residual/homogenization problems, this removes a global scaling ambiguity.

5 Building local MAW blocks

After the fixed ECM rule is known, restrict every local block to the ECM support. For sample j and channel c :

$$A_j^c = [a_{j,e_1}^c \quad a_{j,e_2}^c \quad \cdots \quad a_{j,e_{n_c}}^c], \quad e_k \in Z_{\text{ini}}^c.$$

The target is usually:

$$b_j^c = A_j^c w_{\text{ini}}^c.$$

5.1 Optional sum constraint inside MAW

If the adaptive weights must preserve the sum, augment every local block:

$$\bar{A}_j^c = \begin{bmatrix} A_j^c \\ \mathbf{1}^T \end{bmatrix}, \quad \bar{b}_j^c = \begin{bmatrix} b_j^c \\ S_c \end{bmatrix}.$$

Then all MAW updates solve:

$$\bar{A}_j^c w_j^c = \bar{b}_j^c.$$

This is implemented by:

`_augment_blocks_with_sum_constraint.`

Rule used for hom channels. For homogenization-like output channels, our current strict policy is:

$$A_j^c w_j^c = b_j^c, \quad \mathbf{1}^T w_j^c = \mathbf{1}^T w_{\text{ini}}^c, \quad w_j^c \geq 0.$$

This is professor-aligned: positivity and sum preservation are not optional for hom channels when MAW is active.

6 MAW–ECM pruning problem

At a given pruning state, suppose the current candidate index set is

$$I \subset \{1, \dots, n_c\}.$$

We try to remove one candidate $p \in I$. The tentative support is

$$I_{\text{keep}} = I \setminus \{p\}.$$

For each sample j , the goal is to find new weights on the remaining support:

$$A_{j,I_{\text{keep}}}^c w_{j,I_{\text{keep}}}^{c,\text{new}} = b_j^c, \quad w_{j,I_{\text{keep}}}^{c,\text{new}} \geq 0.$$

The algorithm has two phases.

7 Phase 1: no-regularization least-change pruning

Phase 1 is local in the manifold samples. There is no graph coupling and no explicit active-set solve. For a candidate p , and for each sample j , let

$$w_{j,I_{\text{keep}}}^{\text{before}}$$

be the current weights on the tentative support. We solve for a correction

$$\Delta w_j = \arg \min_{\Delta \in \mathbb{R}^{|I_{\text{keep}}|}} \|\Delta\|_2 \quad \text{s.t.} \quad A_{j,I_{\text{keep}}} \Delta = b_j - A_{j,I_{\text{keep}}} w_{j,I_{\text{keep}}}^{\text{before}}.$$

Equivalently, after the update:

$$w_{j,I_{\text{keep}}}^{\text{after}} = w_{j,I_{\text{keep}}}^{\text{before}} + \Delta w_j, \quad A_{j,I_{\text{keep}}} w_{j,I_{\text{keep}}}^{\text{after}} = b_j.$$

In Python this is:

$$\Delta w_j = \text{np.linalg.lstsq}(A_{j,I_{\text{keep}}}, b_j - A_{j,I_{\text{keep}}} w_j^{\text{before}}).$$

The candidate is accepted if this succeeds for every sample and if the resulting weights are nonnegative:

$$w_j^{\text{after}} \geq -\tau_{\text{neg}}, \quad j = 1, \dots, N_s.$$

Algorithm 1: Phase 1 candidate elimination: no regularization

Input: Blocks A_j, b_j , current weights W , current support I , candidate order.

Output: Accepted support and weights, or failure.

```

1 foreach candidate  $p$  in sorted candidate order do
2    $I_{\text{keep}} \leftarrow I \setminus \{p\}$ 
3   feasible  $\leftarrow$  true
4   for  $j = 1, \dots, N_s$  do
5     Build  $A_{j,I_{\text{keep}}}$ 
6     Check full row rank of  $A_{j,I_{\text{keep}}}$ 
7     Solve the minimum-change least-squares correction  $\Delta w_j$ 
8      $w_j^{\text{after}} \leftarrow w_j^{\text{before}} + \Delta w_j$ 
9     if  $w_j^{\text{after}}$  has negative entries then
10      feasible  $\leftarrow$  false; break
11   if feasible then
12     Set weight of candidate  $p$  to zero for all samples
13     Accept candidate and return
14 Return failure

```

Candidate order. The default order is based on current activity/weight magnitude. Candidates with small total activity are tried first:

$$\text{activity}_e = \sum_{j=1}^{N_s} W_{e,j} \quad \text{or} \quad \max_j |W_{e,j}|$$

depending on whether signed weights are allowed.

8 Stopping Phase 1 and starting Phase 2

There are three ways Phase 2 can start:

1. Phase 1 cannot remove any additional candidate.
2. `smooth_laplacian_all_iterations = 1`, so Phase 2 is forced from the start.
3. A controlled phase-1 stop size is specified:

$$|I| \leq n_{\text{phase1_stop}}.$$

The current flags are:

```
--maw-phase1-stop-size-res,  
--maw-phase1-stop-size-eps,  
--maw-phase1-stop-size-sig.
```

For example, for stress:

```
--maw-phase1-stop-size-sig 20
```

means: use Phase 1 until the stress support reaches 20 candidates, then try Phase 2.

9 Phase 2: positivity enforcement and graph regularization

Phase 2 solves a stronger redistribution problem. It can be local or graph-coupled.

9.1 Stacked variable

Let the current support size be r . Put all sample weights into a matrix

$$W = \begin{bmatrix} | & | & & | \\ w_1 & w_2 & \cdots & w_{N_s} \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{r \times N_s}.$$

Vectorize by samples:

$$z = \text{vec}(W) \in \mathbb{R}^{rN_s}.$$

9.2 Graph regularized least-change objective

Let K_{graph} be the graph Laplacian over manifold samples. The graph Hessian is

$$H = I_{rN_s} + \alpha (K_{\text{graph}} \otimes I_r).$$

The incremental graph update minimizes

$$\frac{1}{2}(z - z_{\text{old}})^T H (z - z_{\text{old}})$$

subject to:

$$A_j z_j = b_j, \quad z_{\text{candidate},j} = 0, \quad z_{a,j} = 0 \text{ for active-set clamped entries.}$$

When $\alpha = 0$, this reduces to a local least-change active-set solve. When $\alpha > 0$, neighboring manifold nodes are encouraged to have coherent weight redistributions.

9.3 Nullspace solve used in Phase 2

For a fixed active set, build one particular feasible point z_p and a nullspace basis N for all equality constraints. Then:

$$z = z_p + Ny.$$

The reduced quadratic problem is:

$$\min_y \frac{1}{2}(z_p + Ny)^T H (z_p + Ny) - g^T (z_p + Ny),$$

where, in incremental mode,

$$g = H z_{\text{old}}.$$

The normal equations are:

$$(N^T H N)y = N^T (g - H z_p).$$

After solving, reconstruct:

$$z_{\text{new}} = z_p + Ny.$$

If any unclamped entry is negative, it is added to the active set and the solve is repeated. This is the explicit positivity-enforcement loop.

Algorithm 2: Phase 2 graph/local active-set pruning for one candidate

Input: Current local weight matrix W , candidate p , blocks A_j, b_j , graph K_{graph} .

Output: Feasible W_{new} , or failure.

```

1 Initialize active sets by clamping candidate  $p$  at every sample
2 for active-set iteration = 1, ...,  $N_{\text{AS,max}}$  do
3   Build particular solution  $z_p$  satisfying all local equalities and clamps
4   Build nullspace basis  $N$ 
5   Solve  $(N^T H N)y = N^T (g - H z_p)$ 
6    $z_{\text{new}} \leftarrow z_p + Ny$ 
7   if all free entries of  $z_{\text{new}}$  are nonnegative then
8     | Accept and return  $W_{\text{new}}$ 
9   | Add negative entries to the active sets
10 Return failure

```

10 Graph construction

Two graph types are currently useful:

Structured grid graph. If the manifold samples are arranged on a structured grid, use the cell connectivity to construct the graph. This is preferred when available because it respects the sampling topology.

KNN graph. If no structured grid exists, build a graph in q -space using nearest neighbors:

$$(j, k) \in \mathcal{E} \quad \text{if} \quad q_k \text{ is among the } K \text{ nearest neighbors of } q_j.$$

Weights can be binary or Gaussian:

$$a_{jk} = \exp \left(-\frac{\|q_j - q_k\|^2}{2\sigma^2} \right).$$

Then

$$K_{\text{graph}} = D - A,$$

where D is the degree matrix and A is the adjacency matrix.

11 RBF surrogate for adaptive weights

After pruning, we have a reduced support Z_{red} and training weights

$$W_{\text{train}} \in \mathbb{R}^{|Z_{\text{red}}| \times N_s}.$$

We fit a multi-output RBF map:

$$q \mapsto \hat{w}(q) \in \mathbb{R}^{|Z_{\text{red}}|}.$$

11.1 Gaussian RBF model

Choose centers $c_k \in \mathbb{R}^{n_q}$, $k = 1, \dots, N_c$, and length scales ℓ . The kernel matrix is:

$$\Phi(q, c_k) = \exp \left(-\frac{1}{2} \sum_{a=1}^{n_q} \frac{(q_a - c_{k,a})^2}{\ell_a^2} \right).$$

The fitted map is:

$$\hat{w}(q) = S [\Phi(q)A + P(q)B]^T,$$

where:

- S is a diagonal output scaling based on the RMS magnitude of each weight.
- $P(q)$ is an optional polynomial block: none, constant, or affine.
- A, B are regression coefficients.

11.2 Regression system

Let

$$Y = (S^{-1}W_{\text{train}})^T.$$

The coefficients solve the regularized least-squares system:

$$\begin{bmatrix} \Phi^T \Phi + \lambda I & \Phi^T P \\ P^T \Phi & P^T P \end{bmatrix} \begin{bmatrix} A \\ B \end{bmatrix} = \begin{bmatrix} \Phi^T Y \\ P^T Y \end{bmatrix}.$$

This is implemented by:

`fit_mawecm_rbf.`

11.3 Online evaluation, clipping, and renormalization

Online evaluation uses:

$$\hat{w}(q) = \text{eval_mawecm_rbf}(q).$$

If positivity is required:

$$\hat{w}_e(q) \leftarrow \max(\hat{w}_e(q), 0).$$

If sum preservation is required:

$$\hat{w}(q) \leftarrow S_c \frac{\hat{w}(q)}{\mathbf{1}^T \hat{w}(q)}.$$

For Newton tangents, the RBF derivative is analytic:

$$\frac{\partial \Phi_k}{\partial q_a} = -\Phi_k(q) \frac{q_a - c_{k,a}}{\ell_a^2}.$$

The derivative of the renormalized weights uses the quotient rule:

$$\frac{\partial}{\partial q_a} \left(S_c \frac{w_e}{\sum_i w_i} \right) = S_c \left[\frac{\partial_a w_e}{\sum_i w_i} - \frac{w_e \sum_i \partial_a w_i}{(\sum_i w_i)^2} \right].$$

This is implemented by:

`eval_mawecm_rbf_with_jacobian.`

12 Offline algorithm

Algorithm 3: Portable MAW–ECM offline build for one channel

Input: Full element data Q^c , sample coordinates q_j , optional graph data.

Output: Support Z_{red}^c , RBF model for $w^c(q)$, diagnostics.

- 1 Build a deterministic classical ECM rule $(Z_{\text{ini}}^c, w_{\text{ini}}^c)$
 - 2 **for** $j = 1, \dots, N_s$ **do**
 - 3 Build local block A_j^c restricted to Z_{ini}^c
 - 4 Set $b_j^c = A_j^c w_{\text{ini}}^c$ or use the full target if desired
 - 5 **if** *sum preservation is required* **then**
 - 6 Augment A_j^c, b_j^c with the row $\mathbf{1}^T w = S_c$
 - 7 Build graph Laplacian K_{graph} if phase 2 graph regularization is enabled
 - 8 Run MAW pruning: Phase 1 first, then Phase 2 if requested or if Phase 1 stalls
 - 9 Fit RBF surrogate $q \mapsto w(q)$ on the remaining support
 - 10 Validate exactness, positivity, sum of weights, and RBF training error
 - 11 Save Z_{red}^c , W_{train}^c , RBF parameters, and diagnostics
-

13 Online algorithm

Algorithm 4: Online HPROM using MAW–ECM weights

Input: Reduced coordinate q , displacement/state decoder, MAW support and RBF model.

Output: Reduced residual and tangent.

- 1 Evaluate state $u(q)$ from the ROM/PROM decoder
 - 2 Evaluate adaptive weights $w(q)$ from the MAW RBF
 - 3 Optionally evaluate dw/dq analytically
 - 4 **foreach** *selected element* $e \in Z_{\text{red}}$ **do**
 - 5 Assemble element residual $r_e(u(q))$ and tangent $K_e(u(q))$
 - 6 Add $w_e(q)r_e$ to the hyper-reduced residual
 - 7 Add $w_e(q)K_e$ to the hyper-reduced tangent contribution
 - 8 If using the weight tangent, add the contribution involving dw/dq
 - 9 Solve the reduced Newton correction
-

Sign convention warning. The extra tangent term from dw/dq depends on the sign convention used by the residual and Newton system in the target codebase. The formal residual derivative is

$$\frac{\partial}{\partial q} \sum_{e \in Z_{\text{red}}} w_e(q)r_e(q) = \sum_{e \in Z_{\text{red}}} \left[w_e(q) \frac{\partial r_e}{\partial q} + r_e(q) \otimes \frac{\partial w_e}{\partial q} \right].$$

When porting to Burgers or another solver, verify the sign by a finite-difference test of the assembled reduced residual.

14 Independent channel strategy

The current recommended design is to keep channels separate:

$$Z_{\text{red}}^{\text{res}}, \quad Z_{\text{red}}^{\text{eps}}, \quad Z_{\text{red}}^{\text{sig}}.$$

The online reduced mesh can be the union:

$$Z_{\text{red}}^{\text{union}} = Z_{\text{red}}^{\text{res}} \cup Z_{\text{red}}^{\text{eps}} \cup Z_{\text{red}}^{\text{sig}}.$$

However, the loops must remain channel-specific:

residual assembly uses $Z_{\text{red}}^{\text{res}}$,

strain output uses $Z_{\text{red}}^{\text{eps}}$,

stress output uses $Z_{\text{red}}^{\text{sig}}$.

Do not loop blindly over the union for every quantity unless the corresponding weights are also defined on the union. The union is a mesh containment device, not necessarily the mathematical cubature rule for every channel.

15 Current mode mapping

Stage8b mode	Adaptive MAW channels	Fixed ECM channels
<code>res_only</code>	residual	eps, sig if <code>ecm_separate</code>
<code>res_eps</code>	residual, eps	sig
<code>res_sig</code>	residual, sig	eps
<code>res_eps_sig</code>	residual, eps, sig	none

The online mode must match the offline file:

Offline file	Online homogenization mode
<code>res_only</code> with fixed hom ECM	<code>ecm_separate</code>
<code>res_sig</code>	<code>sig_maw_eps_ecm</code>
<code>res_eps_sig</code>	<code>maw_separate</code>
<code>res_only</code> diagnostic	<code>full_fom</code>

16 Porting checklist for Burgers 2D ROM Workbench

1. Decide the channels. At minimum use residual. Add output channels only if they are important online.
2. For every training sample, compute element-wise contributions for each channel. The data must be expressible as a sum over elements.
3. Build a deterministic classical ECM rule for each channel.
4. For every channel, build local blocks A_j^c restricted to the ECM support.
5. Set $b_j^c = A_j^c w_{\text{ini}}^c$ for anchor preservation, or set b_j^c to the full integration target if that is more appropriate.
6. Add sum constraints where needed.
7. Enforce positivity where physically/theoretically required.
8. Run MAW pruning:
 - Phase 1: no regularization.
 - Phase 2: local active-set or graph regularization.
9. Fit RBF weight fields for every adaptive channel.
10. Validate:
$$\max_j \frac{\|A_j w_j - b_j\|_2}{\max(\|b_j\|_2, 1)}$$
and check positivity/sum constraints.
11. Integrate $w(q)$ into the online assembler.
12. Verify the reduced tangent against finite differences before trusting Newton iteration counts.

17 Recommended diagnostics

Always save:

- $|Z_{\text{ini}}| \rightarrow |Z_{\text{red}}|$.
- Phase 1 start/end support.
- Phase 2 start/end support, attempts, accepted eliminations.
- Top failure reasons for Phase 2.
- RBF train relative error.
- Constraint residual:

$$\max_j \frac{\|A_j w_j - b_j\|_2}{\max(\|b_j\|_2, 1)}.$$

- Minimum weight.
- Sum-of-weights min/max/mean if sum preservation is active.
- Weight-field plots over the manifold for selected elements.
- Support plots for every channel and for the union.

18 Minimal command pattern in the current code

Residual MAW with classical ECM for homogenization:

```
python3 stage8b_build_mawecm_res_model_rbf.py \  
--dataset-dir stage_8a_mawecm_res_dataset \  
--maw-mode res_only \  
--hom-source ecm_separate \  
--maw-min-support-size-res 0 \  
--use-global-graph-2ndstage 1 \  
--smooth-laplacian-all-iterations 0 \  
--alpha-smooth 1e6 \  
--graph-mode structured_grid \  
--max-number-zeros-active-set-loop-maw-ecm 1 \  
--res-bootstrap-rsvd-randomized 0 \  
--res-bootstrap-constrain-sum-weights 1 \  
--enforce-sum-weights 1 \  
--sum-weights-target 990.0 \  
--tol-rank-rel 1e-14 \  
--save-weight-field-plots 1 \  
--show-weight-field-plots 0 \  
--out-dir stage_8b_hprom_mawecm_res_only_auto_ecmhom_sum990_graph
```

Residual and stress MAW, strain fixed ECM:

```
python3 stage8b_build_mawecm_res_model_rbf.py \
  --dataset-dir stage_8a_mawecm_res_dataset \
  --maw-mode res_sig \
  --hom-source ecm_separate \
  --maw-min-support-size-res 0 \
  --maw-min-support-size-sig 0 \
  --maw-phase1-stop-size-sig 20 \
  --use-global-graph-2ndstage 1 \
  --smooth-laplacian-all-iterations 0 \
  --alpha-smooth 1e4 \
  --graph-mode structured_grid \
  --max-number-zeros-active-set-loop-maw-ecm 1 \
  --res-bootstrap-rsvd-randomized 0 \
  --res-bootstrap-constrain-sum-weights 1 \
  --enforce-sum-weights 1 \
  --sum-weights-target 990.0 \
  --tol-rank-rel 1e-14 \
  --maw-hom-enforce-nonnegativity 1 \
  --maw-hom-conservative 1 \
  --maw-hom-cv-max-rel 0 \
  --save-weight-field-plots 1 \
  --show-weight-field-plots 0 \
  --out-dir stage_8b_hprom_mawecm_res_sig_auto_ecmhom_sum990_graph
```

19 What must change in another codebase

Only the data-building and online assembly layers are codebase-specific:

- Replace Kratos element residual extraction by Burgers element residual extraction.
- Replace q_m source by the Burgers reduced coordinate.
- Replace the Stage8a structured dataset writer.
- Keep the MAW pruning and RBF regression logic almost unchanged.
- Rebuild the online assembler so it evaluates $w(q)$ and loops only over the selected support of the channel being assembled.

20 Common failure modes

No pruning after adding the sum constraint. This usually means the local constraints are too tight for the current support, or the rank tolerance is too conservative. Verify that:

$$|I_{\text{keep}}| \geq m_j$$

for every local block after augmentation.

Phase 2 does nothing. Check the top fail reasons. Typical causes are rank deficiency, too many clamped variables, or infeasible positivity constraints. If a phase-1 stop is too aggressive, try stopping earlier, e.g. 40 instead of 20.

Good constraint fit but bad online behavior. Check the RBF weight fields. Exact offline MAW weights do not guarantee a good interpolant if the fields are rough. Graph regularization and controlled phase-1 stop sizes are meant to improve this.

The HROM mesh has the right union but wrong results. The mesh union is not enough. The online code must map full element ids to local HROM element ids and must use each channel's own support and weights.

21 Summary

The portable MAW–ECM recipe is:

classic ECM $\longrightarrow$ local MAW blocks $\longrightarrow$ Phase 1 pruning
 Phase 2 regularization $\longrightarrow$ RBF $w(q)$ $\longrightarrow$ online weighted assembly.

The key idea is simple: use the nullspace left by a fixed ECM rule to redistribute weights differently at different manifold samples, while preserving the local ECM targets. The final RBF is not a replacement for MAW; it is only the smooth online surrogate of the offline adaptive weights.